

Numerical Methods

Lecture 4: Root Finding Methods

Root Finding - Background

1. Introduction to Root Finding

Root finding is one of the fundamental problems in numerical analysis. Given a function $f(x)$, we seek to find values of x such that:

$$f(x) = 0$$

These values of x are called **roots** (根), **zeros** (零点), or **solutions** (解) of the equation.

1.1 Types of Root Finding Problems

- **Single root** (单根): Function crosses the x-axis at exactly one point
- **Multiple roots** (多根): Function has several intersection points with the x-axis

2. Major Root Finding Algorithms

2.1 Newton's Method (牛顿法)

Formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Advantages: Very fast convergence (quadratic) **Disadvantages:** Requires derivative, may not converge, sensitive to initial guess

2.2 Quasi-Newton Method (准牛顿法)

Principle: Approximate the derivative using finite differences:

$$f'(x_n) \approx \frac{f(x_n + h) - f(x_n)}{h}$$

Advantages: No analytical derivative needed, fast convergence **Disadvantages:** Still sensitive to initial guess

2.3 Secant Method (割线法)

Formula:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Advantages: No derivative needed, good convergence **Disadvantages:** Requires two initial guesses

2.4 Bisection Method (二分法)

Principle: Repeatedly halve the interval containing the root.

Advantages: Always converges, very reliable **Disadvantages:** Slow convergence, requires initial interval

3. Algorithm Selection Guidelines

When choosing which method to use:

- **For guaranteed convergence** (保证收敛): Use bisection method
- **For fastest convergence with smooth functions** (光滑函数最快收敛): Use Newton's method
- **For good balance without derivatives** (无导数的平衡选择): Use quasi-Newton method
- **For functions expensive to evaluate** (函数计算昂贵): Use secant method

General rule: Newton's method is fastest but least reliable, bisection is slowest but most reliable.

4. Common Problems and Failure Modes

4.1 Newton's Method Failures

1. **Division by zero** (除零错误): When $f'(x_n) = 0$ or very close to zero
2. **Wrong root** (错误的根): Converging to a different root than intended
3. **Divergence** (发散): Poor initial guess leading to no convergence

4.2 Multiple Roots Problem (多根问题)

When a function has multiple roots, **different initial guesses may lead to different solutions**. Each root has a **basin of attraction** (吸引域) - the set of initial guesses that will converge to that particular root.

Key insight: Small changes in initial guess can lead to convergence to completely different roots.

4.3 Flat Regions Problem (平坦区域问题)

When $f'(x) \approx 0$ (derivative near zero), Newton's method becomes unstable because:

- The correction term $\frac{f(x)}{f'(x)}$ becomes very large
- Method may "jump" far away from the current position
- Can lead to **numerical overflow** (数值溢出)

4.4 Oscillatory Behavior (振荡行为)

The algorithm can become trapped in a cycle, bouncing between points without converging.

5. Practical Strategies

5.1 For Multiple Roots

- **Plot the function first** to visualize root locations
- **Test multiple initial guesses** systematically
- Use **root bracketing** to identify intervals containing roots

5.2 For Stability Issues

- **Find critical points:** Solve $f'(x) = 0$ to identify problematic regions
- **Avoid starting near flat regions** where $f'(x) \approx 0$
- **Use hybrid approaches:** Start with bisection, then switch to Newton's method

5.3 General Guidelines

- Always **plot the function** before attempting to find roots
- **Test different initial guesses** to understand convergence behavior
- Have a **backup method** (like bisection) for when fast methods fail
- Set **maximum iteration limits** to prevent infinite loops

The key to successful root finding is understanding that **no single method works for all problems** - the choice depends on the specific function and the trade-off between speed and reliability.

2.Code

考试重点： 1.通用函数实现 - 编写一个函数同时支持牛顿法、准牛顿法、割线法 2.收敛性分析 - 分析初始值对收敛性的影响，解释牛顿法失效的原因 3.方法比较 - 比较不同方法的效果和适用性

考试时只需要修改的部分： Module 2 中的函数定义 修改 f(x) 为题目给出的函数 如果需要牛顿法，修改 dfdx(x) 为对应的导数 修改参数： x_min, x_max, initial_guess, initial_values

可选修改： 如果题目要求根区间搜索，修改 search_intervals

不需要修改的部分： Module 1: 所有算法函数 Module 3: 计算逻辑 Module 4: 绘图代码
Module 5, 6: 高级分析（除非题目特别要求）

```
In [ ]: # Module 1: (通常不需要修改)

# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import scipy.optimize as sop

# 先写好各种求根方法的function
# Universal root-finding wrapper: Newton / numerical-derivative Newton /
# - method="newton": uses provided derivative slope(x)
# - method="numdif": uses finite difference derivative approximation
# - method="secant": uses secant slope from two recent iterates
# Returns: (root, iterations)
def newton2(f, x0, method='numdif', atol=1.0e-6, maxiter=100, slope=None,
          assert method in ("newton", "numdif", "secant"))

    # Initialize iterate list:
    # - Secant needs two starting points x0 and x1
    # - Newton/numdif only needs x0
    if method == 'secant':
        x = [x0, x1]
    else:
        x = [x0]

    # Iteration counter
    it = 0
    # Main iteration loop
    while True:
        # True Newton: uses supplied derivative function slope(x)
        if method == 'newton':
            m = 1./slope(x[-1])
        # Quasi-Newton: approximate derivative by forward difference
        elif method == 'numdif':
            m = 1./((f(x[-1]+dx) - f(x[-1])) / dx )
        # Secant: approximate derivative using last two points
        else:
            m = 1./((f(x[-1])-f(x[-2])) / (x[-1]-x[-2]))

        # Update formula (Newton-like step):
        x.append( x[-1] - m * f(x[-1]) )

        # Convergence test: if change in x is below tolerance, stop
        if abs(x[-1]-x[-2]) < atol:
            return x[-1], it

        # Increase iteration count and check maximum iterations
        it += 1
        if it >= maxiter:
            print('hit max its without converging')
            break

# 找到根所在的区间的方法
# Root bracketing:
# Move from a to b in steps of dx until f changes sign.
```

```

# Returns an interval (a_left, a_right) that brackets a root.
def root_bracketing(f, a, b, dx):
    sign = np.sign(f(a))
    while sign == np.sign(f(a)):
        a += dx
        if a >= b:
            raise RuntimeError('no root within [a,b]')
    return (a-dx, a)

# 二分法
# Bisection method:
# Requires an initial bracket [a,b] where f(a) and f(b) have opposite sig
# Iteratively halves the interval until tolerance is met.
def bisection(fct, a, b, atol=1.0E-6, nmax=100):
    n = 0
    while n <= nmax:
        c = (a+b)/2.
        if fct(c) == 0. or (b-a)/2. < atol:
            return c
        n += 1
        if np.sign(fct(c)) == np.sign(fct(a)):
            a = c
        else:
            b = c
    raise RuntimeError('no root found within [a,b]')

# 似牛顿法
# Quasi-Newton method (finite difference derivative):
# Similar to newton2(method="numdif") but returns only the root.
def quasi_newton(f, x0, dx=1.0E-7, atol=1.0E-6):
    x = [x0]
    while True:
        dfdx = (f(x[-1] + dx) - f(x[-1]))/(dx)
        x.append(x[-1] - f(x[-1])/dfdx)
        if abs(x[-1]-x[-2]) < atol:
            return x[-1]

# 割线法
# Secant method:
# Uses two initial guesses and approximates derivative using secant slope
def secant(f, x0, x1, atol=1.0E-6):
    x = [x0, x1]
    while True:
        dfdx = (f(x[-1])-f(x[-2])) / (x[-1]-x[-2])
        x.append(x[-1] - f(x[-1])/dfdx)
        if abs(x[-1]-x[-2]) < atol:
            return x[-1]

# -----

# Module 2: (需要修改)

# 定义函数、导数以及参数, 注意修改
# Define the function whose root(s) you want
# Example here: f(x) = x - exp(-x)
def f(x):
    return x - np.exp(-x)

# 这边方程的导数也需要根据题目改
# Define derivative of f(x) if you want true Newton's method:

```

```

def dfdx(x):
    return 1 + np.exp(-x)

'''x_min, x_max是函数的定义域, initial_guess是初始猜测值, initial_values是收集初
# Domain / initial guesses:
# x_min, x_max: plotting / search range
# initial_guess: a single starting point for quick tests
# initial_values: multiple starting points to test convergence behavior (
x_min, x_max = -2, 3
initial_guess = 0.5
initial_values = [0.3, 0.2, 0.1, 0.0, -0.1, -0.2, -0.5]

'''如果需要具体的区间进行根搜索, 修改这些区间, dx是搜索步长'''
# Optional root bracketing parameters:
# search_intervals: intervals to search for sign changes
# dx: step size for scanning
search_intervals = [(-2, 1), (-1, 1), (0, 2)]
dx = 0.1

# -----

# Module 3 (通常不需要修改)

# 比较多种方法, 并且进行收敛性分析
# Basic method testing with one initial guess
print("=== Basic Method Comparison ===")
x0 = initial_guess
print(f'Initial guess: {x0}')

# SciPy Newton
print('SciPy Newton:', sop.newton(f, x0))
# Our method
print('Our newton2 (numdif):', newton2(f, x0, method='numdif'))

# Test Newton's method if derivative available
try:
    print('Our newton2 (newton):', newton2(f, x0, method='newton', slope=
    print('SciPy Newton (with derivative):', sop.newton(f, x0, dfdx))
except:
    print('No derivative function defined')

# Test secant variant of our newton2
x1 = x0 + 0.1
print('Our newton2 (secant):', newton2(f, x0, method='secant', x1=x1))

# 对不同的初始值做收敛性分析, 每一个初始值都会最后得到一个根, 然后保存起来, 如果遇到重复
# Convergence analysis for many initial values:
# For each initial value, try to find a root and record it.
# Then deduplicate roots (in case different starts converge to same root)
print("\n=== Convergence Analysis with Different Initial Values ===")
roots_found = []

for x0 in initial_values:
    print(f'\nInitial value: {x0}')
    try:
        # SciPy result (as reference)
        root_scipy = sop.newton(f, x0)
        print(f' SciPy: {root_scipy:.6f}')

        # Our quasi-Newton (numdif) + iterations

```

```

    root_quasi, iterations = newton2(f, x0, method='numdif')
    print(f'  Quasi-Newton: {root_quasi:.6f} (iterations: {iterations})

    # Save root for later deduplication and plotting
    roots_found.append(root_scipy)

    # If method fails to converge, print why
    except Exception as e:
        print(f'  Failed to converge: {e}')

# 去掉重复的根, 误差小于1e-6就认为是一样的
# Deduplicate roots (treat roots within 1e-6 as the same)
unique_roots = []
for root in roots_found:
    if not any(abs(root - ur) < 1e-6 for ur in unique_roots):
        unique_roots.append(root)

print(f"\nFound {len(unique_roots)} unique roots: {[f'{r:.6f}' for r in unique_roots]}")

# -----

# Module 4 (通常不需要修改)

# 画图部分
# Create plot and compute dense x-grid for smooth curve
fig, ax1 = plt.subplots(1, 1, figsize=(8, 6))
x = np.linspace(x_min, x_max, 1000)

# Plot function curve and horizontal axis (y=0)
ax1.plot(x, f(x), 'b-', linewidth=2, label='f(x)')
ax1.plot(x, np.zeros_like(x), 'k--', alpha=0.5, label='y=0')

# Mark each unique root on the axis
for i, root in enumerate(unique_roots):
    ax1.plot(root, 0, 'ro', markersize=8, label=f'Root {i+1}: {root:.4f}')

# Set labels, grid, legend, and title
ax1.set_xlabel('$x$', fontsize=16)
ax1.set_ylabel('$f(x)$', fontsize=16)
ax1.grid(True, alpha=0.3)
ax1.legend()
ax1.set_title('Function and its roots')

# Display the plot
plt.show()

# -----

# Module 5: Root Bracketing (附加板块, 根据题目要求加)

# 根区间分析, 就是先找到一个区间 [a,b] 能确定里面至少有一个 root, 然后用这个区间来让...
# Root bracketing analysis (if required by problem)
# Find brackets with sign changes and solve using bisection
print("\n=== Root Bracketing ===")
for (a, b) in search_intervals:
    try:
        # Find a smaller interval inside [a,b] where sign changes
        bracket = root_bracketing(f, a, b, dx)
        print(f'Root bracket in [{a}, {b}]: {bracket}')

```

```

        # Use bisection within the bracket to compute a root
        root_bisection = bisection(f, bracket[0], bracket[1])
        print(f' Bisection result: {root_bisection:.6f}')

    # If no sign change was found, report it
    except RuntimeError as e:
        print(f'No root found in [{a}, {b}]: {e}')

# -----

# Module 6: Numerical Stability Analysis (附加板块, 根据题目要求加)

# 数值稳定性分析, 意思是在小数位精确度有限的情况下、在误差会传播/放大的情况下, 这个算
# Numerical stability analysis (if required by problem)
# Look for problematic regions where  $f'(x) \sim 0$  (flat slope), which can ma
print("\n=== Numerical Stability Analysis ===")

try:
    # Choose a few starting points to search for roots of the derivative
    search_points = np.linspace(x_min, x_max, 10)
    flat_points = []

    # Try to solve  $dfdx(x)=0$  from different starting guesses (SciPy Newton)
    for sp in search_points:
        try:
            flat_point = sop.newton(dfdx, sp)

            # Keep only points where derivative is actually near zero
            if abs(dfdx(flat_point)) < 1e-6:
                flat_points.append(flat_point)
        except:
            continue

    # Remove duplicates among found critical points
    unique_flat_points = []
    for fp in flat_points:
        if not any(abs(fp - ufp) < 1e-4 for ufp in unique_flat_points):
            unique_flat_points.append(fp)

    # Print summary and test root-finding near those points
    print(f"Found {len(unique_flat_points)} critical points where  $f'(x) \approx 0$ ")
    for fp in unique_flat_points:
        print(f" x = {fp:.6f},  $f'(x) = {dfdx(fp):.2e}$ ")

    # Try Newton starting very close to the critical point
    nearby_points = [fp + 0.001, fp - 0.001]
    for np_point in nearby_points:
        try:
            result = sop.newton(f, np_point, dfdx)
            print(f" Starting from {np_point:.6f}: converged to {result:.6f}")
        except:
            print(f" Starting from {np_point:.6f}: failed to converge")

    # Mark critical points on the existing plot
    ax1.axvline(x=fp, color='red', linestyle=':', alpha=0.7,
                label=f'Critical point: {fp:.3f}')

    # Update legend to include critical points if any were found
    if unique_flat_points:
        ax1.legend()

```



```
# If derivative function does not exist, skip stability analysis  
except NameError:  
    print("No derivative function available for stability analysis")  
  
plt.show()
```